

Mujoco システム識別 (SysID / モータ実機 ↔ MuJoCo) について

▼ 元の英語文

<https://x.com/observie/status/2043636045200162876?s=61>

<https://x.com/i/status/2043636045200162876>

System identification (sysid) is the process of finding the physical parameters that make a simulation match reality. If you're training an RL locomotion policy in simulation, the accuracy of your motor model directly affects how well the policy transfers to the real robot.

A recent git commit by

[@kevin_zakka](#)

added a sysid toolbox to MuJoCo which automates this process: you provide recorded motor data and a MuJoCo model, and it optimizes the model parameters to minimize the difference between simulated and real trajectories.

For my

[@RobStride_com](#)

RS02 QDD motors (17 Nm peak, 7.75:1 gear), I built a Rust tool that sends multi-sine torque excitation at 1 kHz and records position/velocity feedback. I then feed this data into MuJoCo's sysid optimizer.

<https://x.com/observie/status/2043647550083797474?s=20>

To identify motor parameters, we need to excite different physical effects at different frequencies:

1. High frequency: the motor reverses direction rapidly. The reflected rotor inertia (armature) resists these fast accelerations.
2. Low frequency: the motor moves slowly, reversing direction gradually. inertia resistance diminishes, while friction becomes dominant.

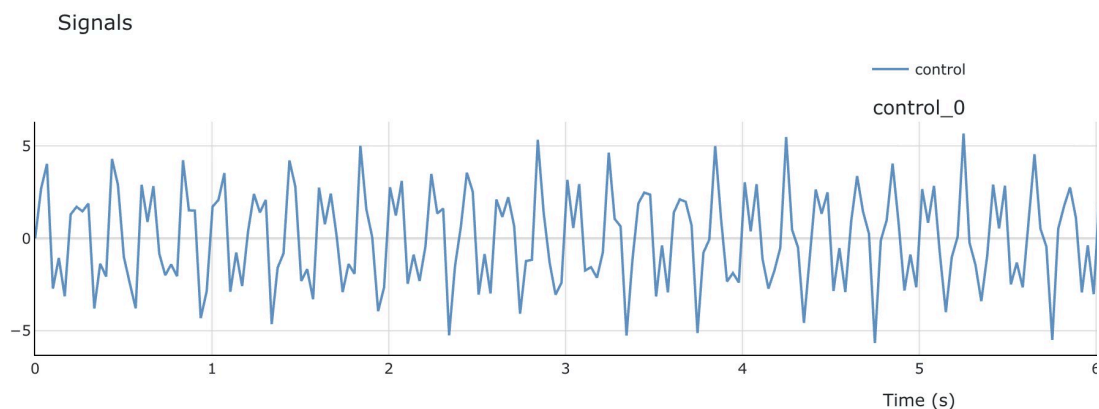
A single sine wave only excites one regime, so I'm using the sum of several frequencies to excite everything in one recording (standard approach in system identification):

$$\text{torque}(t) = \text{amp} \times (\sin(2\pi \cdot f \cdot t) + 0.6 \cdot \sin(2\pi \cdot 3.4f \cdot t) + 0.3 \cdot \sin(2\pi \cdot 7.4f \cdot t))$$

The frequencies are chosen at irrational ratios so they don't cancel each other out. Amplitudes decrease at higher frequencies. I found that base frequency and amplitude matter. Too low freq or too high amp, and the motor builds up speed, hitting the torque-speed curve. Too high freq or too low amp and there's not enough motion for friction to be identifiable. The sweet spot for my RS02 was 4-5 Hz base with 2-3 Nm amplitude.

(plot generated by the mujoco sysid toolbox)

Sequence: rs02 sysid_torque_20260411_103845



<https://x.com/observie/status/2043649950651703714?s=20>

Each sysid recording runs at 1 kHz over a 1 Mbps CAN bus. Every millisecond, I send a torque command (8 bytes + 29-bit extended ID) and ~100 us later the motor begins transmitting its feedback reply: position, velocity, estimated torque, and temperature.

The entire command+reply handshake takes ~300 us, and easily fits a 1ms slot.

I send pure torque commands ($k_p=0$, $k_d=0$) so the motor applies the excitation signal without any internal PD controller masking the physics we're trying to identify.

Each recording produces a CSV with 10,000 timestamped samples over 10 seconds (configurable).



<https://x.com/observie/status/2043650854578704771?s=20>

MuJoCo's sysid toolbox does gray-box identification: you supply the model (a MuJoCo xml) and recorded trajectory data, and the optimizer finds the parameters you designate as unknown.

It replays the same torque commands through the simulated motor and compares predicted position/velocity to the real recordings. The parameters are then adjusted iteratively to minimize the difference.

I used a minimal MuJoCo model of a single hinge joint with a torque actuator, and marked three parameters to identify:

1. armature: reflected rotor inertia through the gearbox
2. frictionloss: Coulomb friction, a constant torque opposing motion
3. damping: viscous friction, proportional to velocity

In the example below I use two 10-second multi-sine trajectories, and the optimizer converges in 16 iterations (a few seconds). The residual drops

from 52.7 to 0.036, a strong indication the simulation now closely follows the real motor's trajectory.

```
david@macbook sysid % uv run identify.py sysid_torque_20260411_103845.csv sysid_torque_20260411_103756.csv
Model: rs02_single.xml (dt=0.001s)
Trajectory 0: sysid_torque_20260411_103845 - 10000 samples, 10.00s, torque [-5.693, 5.693] Nm
Trajectory 1: sysid_torque_20260411_103756 - 10000 samples, 10.00s, torque [-5.693, 5.693] Nm
armature: nominal=0.0042, bounds=[0.0005, 0.05]
frictionloss: nominal=0.1, bounds=[0.0, 2.0]
damping: nominal=0.0, bounds=[0.0, 1.0]
Optimizing with 2 trajectory(s)...
iter: 0 y: 52.69 log10mu: -inf ratio: 0.83 dx: 0.013 reduction: 43
iter: 1 y: 9.199 log10mu: -inf ratio: 0.88 dx: 0.021 reduction: 8.1
iter: 2 y: 1.128 log10mu: -inf ratio: 0.95 dx: 0.022 reduction: 1
iter: 3 y: 0.09196 log10mu: -inf ratio: 0.99 dx: 0.0099 reduction: 0.056
iter: 4 y: 0.03644 log10mu: -inf ratio: 0.9 dx: 0.0082 reduction: 0.00048
iter: 5 y: 0.03596 log10mu: -inf ratio: 2.1 dx: 0.0069 reduction: 2.5e-05
iter: 6 y: 0.03593 log10mu: 0.0 ratio: 0.034 dx: 0.0014 reduction: 9.7e-08
iter: 7 y: 0.03593 log10mu: 0.1 ratio: 0.12 dx: 0.0012 reduction: 2.9e-07
iter: 8 y: 0.03593 log10mu: 0.2 ratio: 1.2 dx: 0.00042 reduction: 6.4e-07
iter: 9 y: 0.03593 log10mu: 1.1 ratio: 0.086 dx: 0.00013 reduction: 2.3e-08
iter: 10 y: 0.03593 log10mu: 4.3 ratio: 0.11 dx: 8.9e-07 reduction: 2.1e-09
iter: 11 y: 0.03593 log10mu: 4.6 ratio: 0.056 dx: 7.2e-07 reduction: 1.4e-09
iter: 12 y: 0.03593 log10mu: 4.7 ratio: 0.88 dx: 4.3e-07 reduction: 9.3e-09
iter: 13 y: 0.03593 log10mu: 5.8 ratio: 0.052 dx: 5.4e-08 reduction: 1e-10
iter: 14 y: 0.03593 log10mu: 5.9 ratio: 1 dx: 2.8e-08 reduction: 6.1e-10
iter: 15 y: 0.03593 log10mu: 6.5 ratio: 0.054 dx: 6.9e-09 reduction: 8.1e-12
Terminated after 16 iterations: insufficient reduction. y: 0.03593, Residual evals: 209
total time 5.1s of which residual 99.9%

armature 0.0042 -> 0.014272
frictionloss 0.1000 -> 0.162646
damping 0.0000 -> 0.000380
```

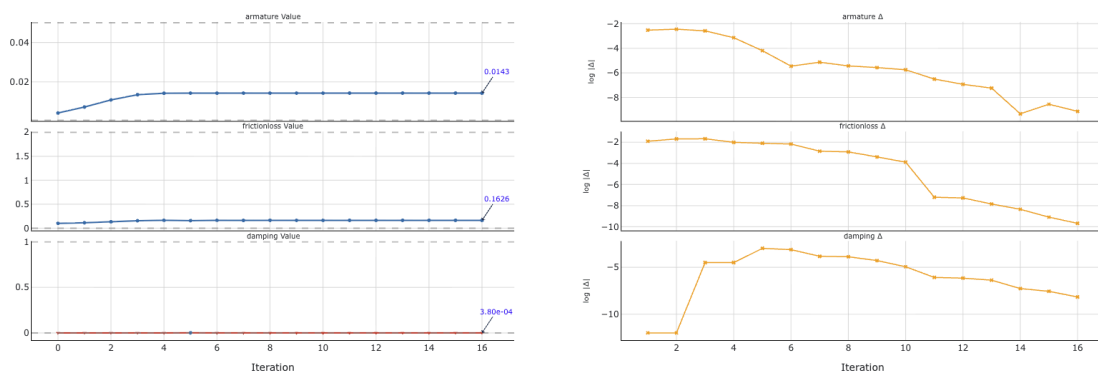
<https://x.com/observie/status/2043651375934967953?s=20>

Results:

1. armature: 0.0042 → 0.0143, much higher than the initial value. Our sim motor was accelerating too fast.
2. frictionloss: 0.1 → 0.163, slightly higher than the initial value.
3. damping: 0 → ~0, no significant viscous friction.

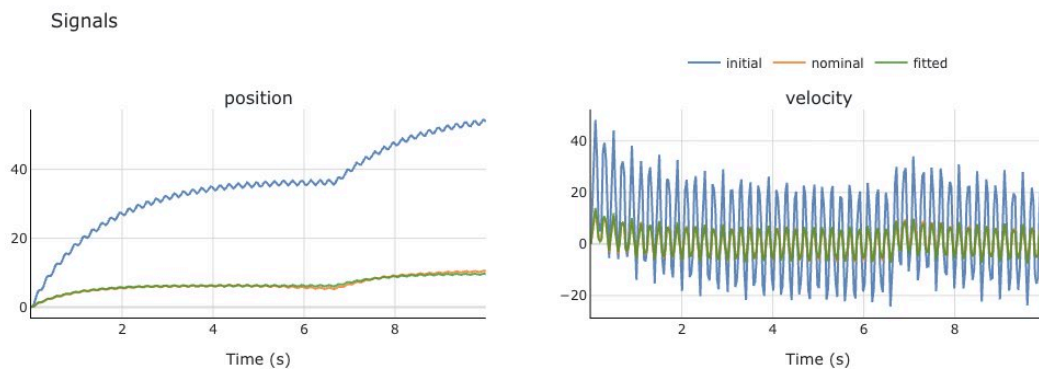
It's important to remember these are not absolute physical ground truths: these are just values that better satisfy the specific model I was using.

(plot generated by the mujoco sysid toolbox)



<https://x.com/observie/status/2043651659507564828?s=20>

These plots demonstrate the effect of sysid: initial model parameters predict high velocities and position values (blue). After adjusting them, the fitted trajectory (green) closely matches the real motor data (orange).

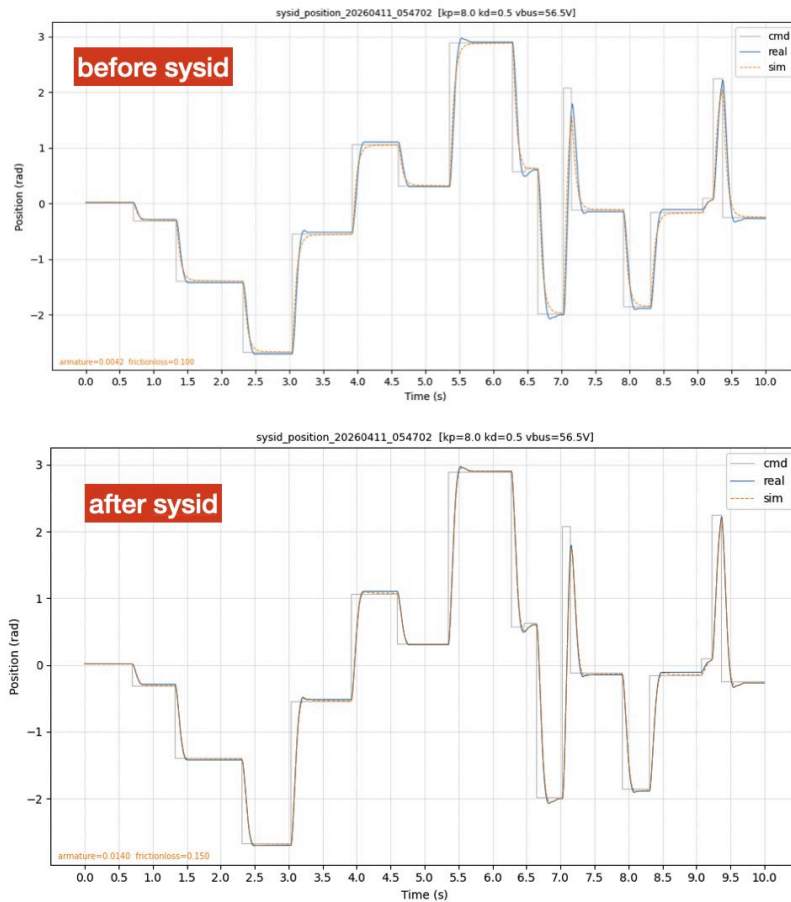


<https://x.com/observie/status/2043653024808095929?s=20>

To validate the identified parameters, I ran a separate test: I sent random position commands to the RS02, configured with a PD controller ($k_p=8$, $k_d=0.5$), recording the motor's position and velocity at a 1 kHz rate over a 10-second period. The motor jumps to random targets between $\pm\pi$ rad while its PD controller tracks the position.

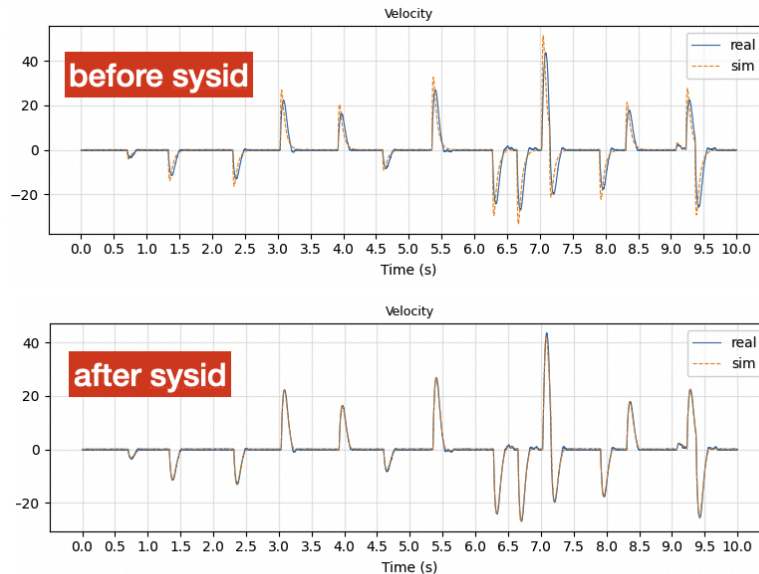
I then fed the same position commands to a MuJoCo actuator model configured with the same K_p/K_d , using both the initial and fitted armature and frictionloss parameters for comparison.

The sim2real gap is visibly reduced using the adjusted parameters: before sysid (top), the sim (orange dashed) overshoots the real motor (blue) at each step, arriving at its targets too quickly. The low inertia makes the simulated motor too responsive. After sysid (bottom), sim and real are nearly indistinguishable.



<https://x.com/observie/status/2043653879275864426?s=20>

The velocity comparison reveals more detail. Before sysid (top), the simulated velocity overshoots the real motor: the sim motor accelerates too strongly due to the underestimated inertia. After sysid (bottom), the velocity traces are nearly identical.



<https://x.com/observie/status/2043654470500782538?s=20>

This was a single RS02 motor on a bench. The identified parameters already make a visible difference in sim-vs-real position tracking. Next up is per-joint sysid on the assembled robot or explore other mujoco models, such as the new dcmotor actuator model (just added by

[@yuvaltassa](#)

). The more accurate the simulation, the less we need to rely on DR. Ideally we would use DR mainly for inherently stochastic data (e.g., noisy sensor data).

As always in this robots building journey log, I am sharing things I just learnt, so please send me your feedback. What did I miss? What did I get wrong?
Thanks

質問 1

Excuse me, what is MuJoCo's sysid toolbox?

Check it out:https://x.com/kevin_zakka/status/2022351158883610719?s=20

SysID intro Colab notebook:<https://t.co/ZDUZNWY9NZ>

質問 2

<https://x.com/valenorf/status/2022366199640703001?s=20>

Question about the sysid. Runs on mujoco warp as well for usage in mjlab?

https://x.com/kevin_zakka/status/2022394754890371176?s=20

The sysID toolbox is powered by C mujoco, not mjwarp. But it's multi-threaded and fast. Once you have an identified model, you can just plug that model into mjwarp/mjlab.

<https://x.com/valenorf/status/2022424764267270200?s=20>

I was curious about possible sim-to-sim gap between mj warp and mujoco, so identifying in one could be not the same on the other. But if they are very close C mujoco is perfect

質問 3

<https://x.com/GoingBallistic5/status/2043655436121878935?s=20>

Shouldn't the manufacturer provide those parameters with the actuator?

Have you tested "identical" actuators to see if they match?

<https://x.com/observie/status/2043658249996542324?s=20>

Manufacturers do usually provide those params (see

[@RobStride_com](#)'s here

<https://x.com/observie/status/2035294794642059390>

).

Having said that, the parameters we find via sysid are not the physical ground truth parameters: they are the parameters that make our selected simulator model work the best. So it's not a number manufacturers can provide: it's a number you tune for your specific selected motor model.

I have not tested several actuators of the same model, because, quite frankly, even two runs of the same specific unit produce slightly different values. Additionally, I expect that the no-load motor values be just the

baseline: per-joint sysid on the assembled robot will pick in additional detail not detectable on a bench motor.

<https://x.com/observie/status/2035294794642059390?s=20>

[@RobStride_com](#)

モーターを使った sim2real をやっていますか？

彼らの Discord チャンネルに埋もれていたこの宝石を見つけました - モーターの慣性モーメント、出力側（低速端）：

RS00: 0.001 kg·m²

RS01 : 0.0042 kg·m²

RS02 : 0.0042 kg·m²

RS03 : 0.02 kg·m²

RS04 : 0.04 kg·m²

RS05 : 0.0007 kg·m²

RS06 : 0.012 kg·m²

<https://x.com/observie/status/2035336897728262176?s=20>

Found some more goodies

← # 【🧠】 ml-general



Ali K 08/08/2025, 21:09

here are the friction values we use. you can normally get it from the metadata in the kscales api, but that is down because we are refactoring the code atm

```
robstride_00
-----
max_torque: 14
armature: 0.001
frictionloss: 0.1
max_velocity: 27.227

robstride_02
-----
max_torque: 17
armature: 0.0042
frictionloss: 0.1
max_velocity: 37.699

robstride_03
-----
max_torque: 60
armature: 0.02
frictionloss: 0.2
max_velocity: 18.849

robstride_04
-----
max_torque: 120
armature: 0.04
frictionloss: 0.2
max_velocity: 17.488
```

[Jump to Message](#)

 **@Kayden Knapik** No I havent. Figured since im using exact same motors as kscale I could just use ...

 **scott** 08/08/2025, 18:35
Those inertia and friction values came direct from the vendor. (edited)

 **@scott** Those inertia and friction values came direct from the vendor.

 **Kayden Knapik** 08/08/2025, 18:37
From robstride?
Is it on the product information?

 **@Kayden Knapik** From robstride?

 **scott** 08/08/2025, 18:38
Yes. I believe we just asked them and they replied with the values, don't think it was in the literature.

 1

<https://x.com/observie/status/2035342571694628985?s=20>

This also shows up in the kscale repositories (e.g. metadata.json):

```
"robstride_03": {  
  "actuator_type": "robstride_03",  
  "sysid": "",  
  "max_torque": 60.0,  
  "armature": 0.02,  
  "damping": null,  
  "frictionloss": 0.2,  
  "vin": null,  
  "kt": null,  
  "R": null,
```

```

"vmax": null,
"amax": null,
"max_velocity": 18.849,
"max_pwm": null,
"error_gain": null
},
"robstride_02": {
"actuator_type": "robstride_02",
"sysid": "",
"max_torque": 17.0,
"armature": 0.0042,
"damping": null,
"frictionloss": 0.1,
"vin": null,
"kt": null,
"R": null,
"vmax": null,
"amax": null,
"max_velocity": 37.699,
"max_pwm": null,
"error_gain": null
},
"robstride_00": {
"actuator_type": "robstride_00",
"sysid": "",
"max_torque": 14.0,
"armature": 0.001,
"damping": null,
"frictionloss": 0.1,
"vin": null,
"kt": null,
"R": null,
"vmax": null,
"amax": null,
"max_velocity": 27.227,
"max_pwm": null,
"error_gain": null
},
"robstride_04": {

```

```
"actuator_type": "robstride_04",  
"sysid": "",  
"max_torque": 120.0,  
"armature": 0.04,  
"damping": null,  
"frictionloss": 0.2,  
"vin": null,  
"kt": null,  
"R": null,  
"vmax": null,  
"amax": null,  
"max_velocity": 17.488,  
"max_pwm": null,  
"error_gain": null  
}
```

<https://x.com/GoingBallistic5/status/2043661061560074417?s=20>

The two run differences is likely temperature or wear and tear.

Can you add a temperature parameter to the model, or do you need to swap the sysids for warm or cold states?

<https://x.com/observie/status/2043664117504840064?s=20>

I suspect that not only temperature, but also voltage variation can affect the result. I don't think the standard MuJoCo actuator model can account for either of those, but MuJoCo's new dcmotor actuator model (just merged) seems to have modeled both. Haven't tested it yet but it seems promising.

Re two run differences: I used relatively short trajectories (10 seconds) so maybe they're not long enough to pack the entire potential data, hence the small variations. Maybe if I use additional trajectories of longer durations, the variation in the result will

<https://x.com/observie/status/2043668001791054031?s=20>

The main difference between two different trajectories of the same excitation function, under the same voltage/temperature conditions, is the sensor noise: each trajectory will read slightly different sensor results. I therefore suspect that the longer we make the trajectory, the more the noise would average out

▼ 準翻訳（段落ごと）

以下、ページ本文（折りたたみ「元の英語文」の外側にある本文）を**段落（= 各ポスト相当）**ごとに、内容を落とさない形で自然な日本語にしています。専門用語は、あとで検索しやすいよう **英語も併記**しています。

1) System identification とは (sysid)

システム同定 (system identification, **sysid**) とは、**シミュレーションが現実**に**一致する**ように、物理パラメータを見つけるプロセスです。

もしシミュレーション上で RL の歩行 (locomotion) ポリシーを学習しているなら、**モータモデルの精度**は、そのポリシーが実機ロボットにどれだけうまく転移するか (sim2real) に直接効いてきます。

最近、@kevin_zakka による git のコミットで、MuJoCo に **sysid 用ツールボックス (sysid toolbox)** が追加されました。

やることは、**記録したモータデータと MuJoCo モデル**を与えると、シミュレーションと実機の軌道差が最小になるように、モデルのパラメータを自動で最適化してくれる、というものです。

私の RobStride の RS02 QDD モータ（ピークトルク 17 Nm、減速比 7.75:1）では、1kHz でマルチサイン (multi-sine) のトルク励振を送って、位置・速度のフィードバックを記録する Rust ツールを作りました。そのデータを MuJoCo の sysid 最適化に入力しています。

2) どの周波数で何が同定できるか (multi-sine の意義)

モータのパラメータを同定するには、**周波数帯によって違う物理効果**を励起する必要があります。

- **高周波**：モータが高速に正逆転します。ギアを介して見えるロータ慣性 (reflected rotor inertia, **armature**) が、こうした速い加速度変化に抵抗します。
- **低周波**：モータがゆっくり動いて徐々に反転します。この領域では慣性の影響は弱まり、代わりに**摩擦 (friction)** が支配的になります。

単一の正弦波（sine wave）だとどちらか片方の領域しか励起できないので、私は複数周波数を足し合わせて、**1回の記録で全部を励起する**（システム同定の標準的手法）ようにしています：

```
torque(t) = amp × (sin(2π·f·t) + 0.6·sin(2π·3.4f·t) + 0.3·sin(2π·7.4f·t))
```

周波数は互いに打ち消し合わないよう、**無理数比（irrational ratios）**になるよう選んでいます。高周波ほど振幅は小さくしています。

また、基準周波数（base frequency）と振幅（amplitude）の選び方は重要でした。

- 周波数が低すぎる／振幅が大きすぎると、モータが速度を乗せてしまい、トルク-速度特性（torque-speed curve）に当たってしまう。
- 周波数が高すぎる／振幅が小さすぎると、摩擦が同定できるほど十分に動かない。

私の RS02 では、**基準周波数 4～5 Hz、振幅 2～3 Nm**あたりがちょうど良い落とし所でした。

（プロットは MuJoCo sysid toolbox が生成）

3) 記録（logging）の条件：1kHz / CAN / トルク指令

それぞれの sysid 用の記録は、**1 Mbps の CAN バス**上で **1 kHz** で動かしています。毎ミリ秒、私はトルクコマンド（8バイト + 29-bit の拡張ID）を送ります。するとおよそ **100 μs** 後に、モータがフィードバック応答を送り始めます（位置、速度、推定トルク、温度）。

このコマンド+応答のやり取り全体は **約 300 μs**で完了し、1ms の枠内に十分収まります。

さらに私は、モータ内部の PD 制御が「同定したい物理」を隠してしまわないように、**純粋なトルク指令**（ $k_p=0, k_d=0$ ）を送って、励振信号がそのまま物理に効くようにしています。

各記録は 10 秒間（設定可）で、10,000 サンプルの CSV（タイムスタンプ付き）になります。

4) MuJoCo sysid toolbox の同定方式（gray-box）と未知パラメータ

MuJoCo の sysid toolbox は **灰色箱同定（gray-box identification）** です。つまり、MuJoCo の XML モデルと実測の軌道データを与えると、あなたが「未知」と指定したパラメータを、最適化で推定してくれます。

具体的には、同じトルクコマンドをシミュレーション側のモータに再生し、予測された位置・速度を実測と比較します。そして差が小さくなるようにパラメータを反復的に調整します。

私は「単一ヒンジ関節＋トルクアクチュエータ」の最小モデルを使い、次の3つを同定対象にしました：

1. **armature**：ギアを通して見えるロータの反射慣性（reflected rotor inertia）
2. **frictionloss**：クーロン摩擦（Coulomb friction）。運動に逆らう一定トルク
3. **damping**：粘性摩擦（viscous friction）。速度に比例する摩擦

下の例では、10秒のマルチサイン軌道を2本使い、最適化は 16 イテレーション（数秒）で収束しました。残差（residual）は 52.7 から 0.036 に低下しており、**シミュレーションが実機の軌道をかなりよく追うようになった強いサイン**です。

5) 同定結果（例）：armature / frictionloss / damping

結果は以下でした：

1. **armature**：0.0042 → 0.0143（初期値よりかなり大きい）。シミュレーションのモータが加速しすぎていた。
2. **frictionloss**：0.1 → 0.163（初期値より少し大きい）

3. **damping** : 0 → ほぼ 0 (有意な粘性摩擦は見えない)

重要なのは、これらが「絶対的な物理的真値」ではない点です。これは、私が選んだ特定のモータモデル／シミュレータ設定に対して、より良く成立する値、という意味です。

(プロットは MuJoCo sysid toolbox が生成)

6) sysid の効果 (プロットの意味)

これらのプロットは sysid の効果を示しています。

初期パラメータでは、速度や位置が (青で示される) 実機データとずれて予測されていました。パラメータ調整後は、フィットした軌道 (緑) が実測 (オレンジ) とかなり近く一致します。

7) 妥当性検証：別の試験 (ランダム位置指令 + PD) で比較

同定したパラメータを検証するために、別のテストも行いました。

RS02 に対してランダムな位置指令を送り、PD 制御 ($k_p=8$, $k_d=0.5$) で追従させた状態で、10秒間 1kHz で位置と速度を記録しました。目標位置は $\pm\pi$ rad の範囲でランダムに飛びます。

次に、同じ位置指令を MuJoCo 側のアクチュエータモデルにも与え、同じ K_p/K_d で、**初期パラメータ**と **sysid 後パラメータ (armature と frictionloss)** の両方を比較しました。

sysid 後のパラメータを使うと、sim2real のギャップが目に見えて減ります。

sysid 前 (上段) では、シミュレーション (オレンジ破線) が実機 (青) よりも各ステップでオーバーシュートし、目標に到達するのが速すぎます。低い慣性のせいでシミュレーションのモータが過敏だったわけです。

sysid 後 (下段) では、シミュレーションと実機がほぼ見分けがつかないくらい一致します。

8) 速度比較（より詳細に見える差）

速度の比較を見ると、さらに分かりやすいです。

sysid 前（上段）では、シミュレーションの速度が実機をオーバーシュートしています。これは慣性が過小評価されていたため、シミュレーションのモータが強く加速しすぎたからです。

sysid 後（下段）では、速度波形もほぼ一致します。

9) 今後：単体ベンチ → ロボット全体 / dcmotor モデル / DR との関係

今回の実験は、ベンチ上の RS02 モータ1台だけでした。それでも、同定したパラメータによって sim-vs-real の位置追従が目に見えて改善することが分かりました。

次は、組み上げたロボット上での **関節ごとの sysid (per-joint sysid)** をやるか、あるいは新しい MuJoCo モデル（たとえば新しく追加された **dcmotor actuator model**）を試したいです。

シミュレーションが正確になればなるほど、DR（domain randomization）に頼る必要は減ります。理想的には、DR は本質的に確率的な要素（例：センサノイズ）のために主に使い、それ以外はモデル精度で詰められると良いです。

<https://x.com/i/status/2043636045200162876>

研究で使う用の要点まとめ（SysID / モータ実機 ↔ MuJoCo）

目的（何が嬉しいか）

- **sim2real ギャップ低減**：歩行などの RL ポリシーをシミュレーションで学習する場合、**モータモデル精度**が実機転移性能を左右する。
- **System identification (sysid)** で、実機ログと MuJoCo モデルの差が最小になるように、モデルパラメータを同定していく。

MuJoCo SysID toolbox の位置づけ（何をするツールか）

- **gray-box identification（灰色箱同定）**：
 - 既知：モデル構造（MuJoCo XML、関節・アクチュエータ構成など）
 - 未知：指定したパラメータ（例：armature / frictionloss / damping）
- 実機で与えたのと同じ入力（例：トルク指令）をシミュレーションで再生し、**位置・速度の軌道差**が最小になるように未知パラメータを最適化。

実機ログ取得（この例での要点）

- **制御入力**：純トルク指令（ $k_p=0, k_d=0$ ）
 - 内部 PD が物理特性を「隠す」ことを避けるため。
- **サンプリング**：1 kHz
- **通信**：1 Mbps CAN
- **フィードバック**：位置・速度・推定トルク・温度（例）
- **1回の記録**：10秒・10,000サンプルの CSV（設定可）

入力設計（multi-sine を使う理由・設計指針）

- 同定したい物理効果は周波数で分離されやすい：
 - **高周波**：反転が速い → 反射慣性（reflected rotor inertia, **armature**）が効く
 - **低周波**：ゆっくり反転 → 慣性は効きづらくなり、**摩擦（friction）**が支配的
- 単一正弦波は1領域しか励起しがちなので、複数周波数の和（**multi-sine**）で1回の記録に「全部」を含める。

- 例：
 - $\text{torque}(t) = \text{amp} \times (\sin(2\pi \cdot f \cdot t) + 0.6 \cdot \sin(2\pi \cdot 3.4f \cdot t) + 0.3 \cdot \sin(2\pi \cdot 7.4f \cdot t))$
 - 周波数比は打ち消しを避けるため**無理数比**っぽく選ぶ
 - 高周波ほど振幅を小さくする
- チューニング観点（例）：
 - 周波数が低すぎる／振幅が大きすぎる → 速度が乗ってトルク-速度特性に入りやすく同定が崩れる
 - 周波数が高すぎる／振幅が小さすぎる → 動きが小さく摩擦が同定しにくい
 - 例の sweet spot：基準周波数 4～5 Hz、振幅 2～3 Nm

同定対象パラメータ（最低限セットの例）

- **armature**：ギア越しの反射慣性（モータが「加速しやすい/しにくい」を決める）
- **frictionloss**：クーロン摩擦（一定トルクで運動に逆らう）
- **damping**：粘性摩擦（速度比例）

結果の読み方（重要な注意）

- 同定値は「物理の真値」ではなく、選んだ MuJoCo のモデル構造の中で最も整合する値。
- 同じ個体でも、ログの違い（ノイズ等）で値は多少揺れる。

妥当性検証（validation）のやり方例

- sysid に使った軌道とは別のテストで、**再現性（汎化）**を確認する。
- 例：ランダム位置目標を PD ($k_p=8, k_d=0.5$) で追従させたログ（位置・速度）を取り、
 - シミュレーション（同じ K_p/K_d ）で「初期パラメータ」と「同定後パラメータ」を比較

- sysid 後に、オーバーシュートや立ち上がり差が減るか（位置・速度どちらも）を見る

研究に落とすときのチェックリスト（そのまま ToDo 化できる形）

- ☐ 対象関節（単体/組付け後）を決める（ベンチ vs. 実機組付け後で同定値が変わり得る）
- ☐ ログ仕様を固定する（サンプルレート、入力種類、記録時間、温度・電圧の取り扱い）
- ☐ multi-sine の f/amp を設計し、速度が乗りすぎない範囲に調整する
- ☐ MuJoCo 側の最小モデルから始め、同定対象パラメータ（armature/frictionloss/damping など）を指定する
- ☐ 収束指標（残差）と、別軌道での validation をセットで評価する

発展メモ（この投稿で触れられている次の方向性）

- 温度・電圧変動が同定値に影響し得る。
- 標準アクチュエータモデルでは表現しきれない可能性があり、**dcmotor actuator model** など別モデルも検討余地。